

Web Services & SOA (IT644)

Outline

- 1 Understanding Web Services
- 2 More on Service-oriented Architecture (SOA)
- 3 Microservices Model

Defining Web Service

A web service (WS) is either:

- a service offered by an electronic device to another electronic device, communicating with each other via the Internet, or
- a server running on a computer device, listening for requests at a particular port over a network, serving web documents (HTML, JSON, XML, images).

In a web service, a web technology such as HTTP is used for transferring machine-readable file formats such as XML and JSON.

What is a Web Service?

- Web services are a type of internet software that use standardized messaging protocols and are made available from an application service provider's web server for a client or other web-based programs to use.
- These services are sometimes referred to as web application services.
- They provide powerful, flexible interoperability, enabling machine-to-machine interactions across a network even with machines and software stacks not designed to work together natively.
- Web services range from major services such as storage management and customer relationship management to more limited services such as furnishing a stock quote and checking bids for an auction item.

Key differences / concepts

- Web Application is a complete Application with a Graphical User Interface (GUI), however, web services do not necessarily have a user interface since it is used as a component in an application.
- Web services offer a shared platform that enables the communication between diverse applications created using different programming languages.
- In cloud computing, web services are searchable and callable across a network.

What is service-oriented architecture (SOA)?

- SOA is a method of software development that uses software components called services to create business applications.
- Each service provides a business capability, and services can also communicate with each other across platforms and languages.
- Developers use SOA to reuse services in different systems or combine several independent services to perform complex tasks.

What is service-oriented architecture? ...cont'd

- For example, multiple business processes in an organization require the user authentication functionality.
- Instead of rewriting the authentication code for all business processes, you can create a single authentication service and reuse it for all applications.
- Similarly, almost all systems across a healthcare organization, such as patient management systems and electronic health record (EHR) systems, need to register patients. These systems can call a single, common service to perform the patient registration task.

Basic Architectural Model of Web Services

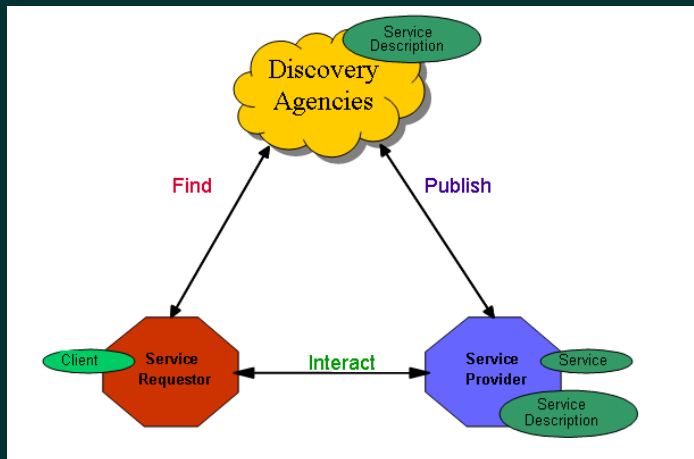


Figure: Architecture of Web Services

Benefits of SOA over Monolithic Architectures

Faster time to market

Developers reuse services across different business processes to save time and costs. They can assemble applications much faster with SOA than by writing code and performing integrations from scratch.

Efficient maintenance

It's easier to create, update, and debug small services than large code blocks in monolithic applications. Modifying any service in SOA does not impact the overall functionality of the business process.

Greater adaptability

SOA is more adaptable to advances in technology. You can modernize your applications efficiently and cost effectively. For example, healthcare organizations can use the functionality of older electronic health record systems in newer cloud-based applications.

Basic principles of SOA

There are no well-defined standard guidelines for implementing service-oriented architecture (SOA). Some basic principles are common across all SOA implementations.

Interoperability

Each service in SOA includes description documents that specify the functionality of the service and the related terms and conditions. Any client system can run a service, regardless of the underlying platform or programming language. For instance, business processes can use services written in both C and Python. Since there are no direct interactions, changes in one service do not affect other components using the service.

Granularity

Services in SOA should have an appropriate size and scope, ideally packing one discrete business function per service. Developers can then use multiple services to create a composite service for performing complex operations.

Basic principles of SOA ... cont'd

Loose coupling

Services in SOA should be loosely coupled, having as little dependency as possible on external resources such as data models or information systems. They should also be stateless without retaining any information from past sessions or transactions. This way, if you modify a service, it won't significantly impact the client applications and other services using the service.

Abstraction

Clients or service users in SOA need not know the service's code logic or implementation details. To them, services should appear like a black box. Clients get the required information about what the service does and how to use it through service contracts and other service description documents.

Components in service-oriented architecture(SOA)

There are 4 main components in SOA - Service, Service provider, Service consumer, & Service registry

- **Service** - They are the basic building blocks of SOA. They can be **private** - available only to internal users of an organization, or **public** - accessible over the internet to all. Individually, **each service has three main features**.
 - **Service implementation**
The service implementation is the code that builds the logic for performing the specific service function, such as user authentication or bill calculation.
 - **Service contract**
The service contract defines the nature of the service and its associated terms and conditions, such as the prerequisites for using the service, service cost, and quality of service provided.
 - **Service interface** In SOA, other services or systems communicate with a service through its service interface. The interface defines how you can invoke the service to perform activities or exchange data. It reduces dependencies between services and the service requester. For example, even users with little or no understanding of the underlying code logic can use a service through its interface.

Components in SOA ... cont'd

- **Service provider**

The service provider creates, maintains, and provides one or more services that others can use. Organizations can create their own services or purchase them from third-party service vendors.

- **Service consumer**

The service consumer requests the service provider to run a specific service. It can be an entire system, application, or other service. The service contract specifies the rules that the service provider and consumer must follow when interacting with each other. Service providers and consumers can belong to different departments, organizations, and even industries.

- **Service registry** A service registry, or service repository, is a network-accessible directory of available services. It stores service description documents from service providers. The description documents contain information about the service and how to communicate with it. Service consumers can easily discover the services they need by using the service registry.

How does service-oriented architecture work?

- In SOA, services function independently and provide functionality or data exchanges to their consumers.
- The consumer requests information and sends input data to the service.
- The service processes the data, performs the task, and sends back a response.
- For example, if an application uses an authorization service, it gives the service the username and password. The service verifies the username and password and returns an appropriate response.

Communication protocols to implement SOA

- Services communicate using established rules that determine data transmission over a network. These rules are called communication protocols.
- Some standard protocols to implement SOA include the following:
 - Simple Object Access Protocol (SOAP)
 - RESTful HTTP
 - Apache Thrift
 - Apache ActiveMQ
 - Java Message Service (JMS)

Remember

You can even use more than one protocol in your SOA implementation.

What is an ESB in SOA?

- An enterprise service bus (ESB) is software that you can use when communicating with a system that has multiple services.
- It establishes communication between services and service consumers no matter what the technology.
- Benefits of ESB
 - An ESB provides communication and transformation capabilities through a reusable service interface.
 - You can think of an ESB as a centralized service that routes service requests to the appropriate service.
 - It also transforms the request into a format that is acceptable for the service's underlying platform and programming language.

Limitations in implementing SOA

- **Limited scalability**

System scalability is significantly impacted when services share many resources and need to coordinate to perform their functionality.

- **Increasing interdependencies**

Service-oriented architecture (SOA) systems can become more complex over time and develop several interdependencies between services. They can be hard to modify or debug if several services are calling each other in a loop. Shared resources, such as centralized databases, can also slow down the system.

- **Single point of failure**

For SOA implementations with an ESB, the ESB creates a single point of failure. It is a centralized service, which goes against the idea of decentralization that SOA advocates. Clients and services cannot communicate with each other at all if the ESB goes down.

What are microservices?

- Microservices architecture is made up of very small and completely independent software components, called microservices, that specialize and focus on one task only.
- Microservices communicate through APIs, which are rules that developers create to let other software systems communicate with their microservice.
- The microservices architectural style is best suited to modern cloud computing environments.
- They often operate in containers—independent software units that package code with all its dependencies.

Benefits of microservices

- Microservices are independently scalable, fast, portable, and platform agnostic—characteristics native to the cloud.
- They are also decoupled, which means they have limited to no dependencies on other microservices.
- To achieve this, microservices have local access to all the data they need instead of remote access to centralized data that other systems also access and use.
- This creates data duplication which microservices make up for in performance and agility.

SOA compared to microservices

- Microservices architecture is an evolution of the SOA architectural style.
- Microservices address the shortcomings of SOA to make the software more compatible with modern cloud-based enterprise environments.
- They are fine grained and favor data duplication as opposed to data sharing.
- This makes them completely independent with their own communication protocols that are exposed through lightweight APIs.
- It's essentially the consumers' job to use the microservice through its API, thus removing the need for a centralized ESB.